

jepctl: A Local Runtime for Joint-Embedding Predictive Architectures, with Few-Shot Perception, an Online Latent World Model, and Robot Control

Younss

Independent researcher
younssiga@gmail.com

September 21, 2026

Abstract

Joint-Embedding Predictive Architectures (JEPAs) learn by predicting representations rather than pixels, and a growing body of work shows that the resulting latent spaces support recognition, anomaly detection, and control. That capability is largely confined to research code: running a JEPA encoder on a laptop, against a live camera or microphone, and wiring it to an application still requires assembling a Python stack by hand. We present `jepctl`, a single-binary local runtime that makes JEPA-style encoders operational the way `ollama` made local language models operational. It loads published checkpoints with strict, auditable weight coverage; exposes embeddings over a REST and SSE API and a command line; and ships four worked applications that use only latent-space operations: (i) few-shot visual and audio recognition from a handful of captures, with a per-patch divergence map that localises the deviation; (ii) an online latent world model over the live camera that predicts the next embedding and reports its prediction error as a surprise signal, following the design of LeWorldModel [10]; (iii) a robot arm driven either by continuous posture mirroring or by minimising a latent energy toward a goal image, with no trajectory programming and no reward; and (iv) a multimodal companion that fuses vision and audio locally at 30 Hz. Everything runs on device: no cloud, no transcription model, and no object detector. We report single-machine measurements on an Apple M5 Pro, including full checkpoint coverage for six published encoders, per-frame latencies of 12–46 ms for a DINOv2-S front end, and a peak resident set of 1.98 GB for a V-JEPA 2 clip forward after removing an autograd-graph retention bug. We also state plainly what this class of model does not provide: it is not generative and it does not produce metric depth. `jepctl` is Apache-2.0 licensed and available at <https://github.com/younss/jepctl>.

1 Introduction

A predictive model of the world is useful only if something can run it. The JEPA line of work [9, 1, 2] argues that an agent should learn by predicting *representations* of what it has not yet seen, rather than reconstructing pixels, and recent results extend that principle to end-to-end training from raw pixels [10] and to domains far from vision [3]. What has not kept pace is the path from a published checkpoint to a working local system. Language models crossed that gap when single-binary runtimes made “pull a model, run it, call an API” a one-line operation [11]. Representation learning has no equivalent: a practitioner who wants to use I-JEPA or V-JEPA 2 against their own webcam assembles a Python environment, writes their own preprocessing, and discovers only at inference time whether the checkpoint layout they downloaded matches the architecture they built.

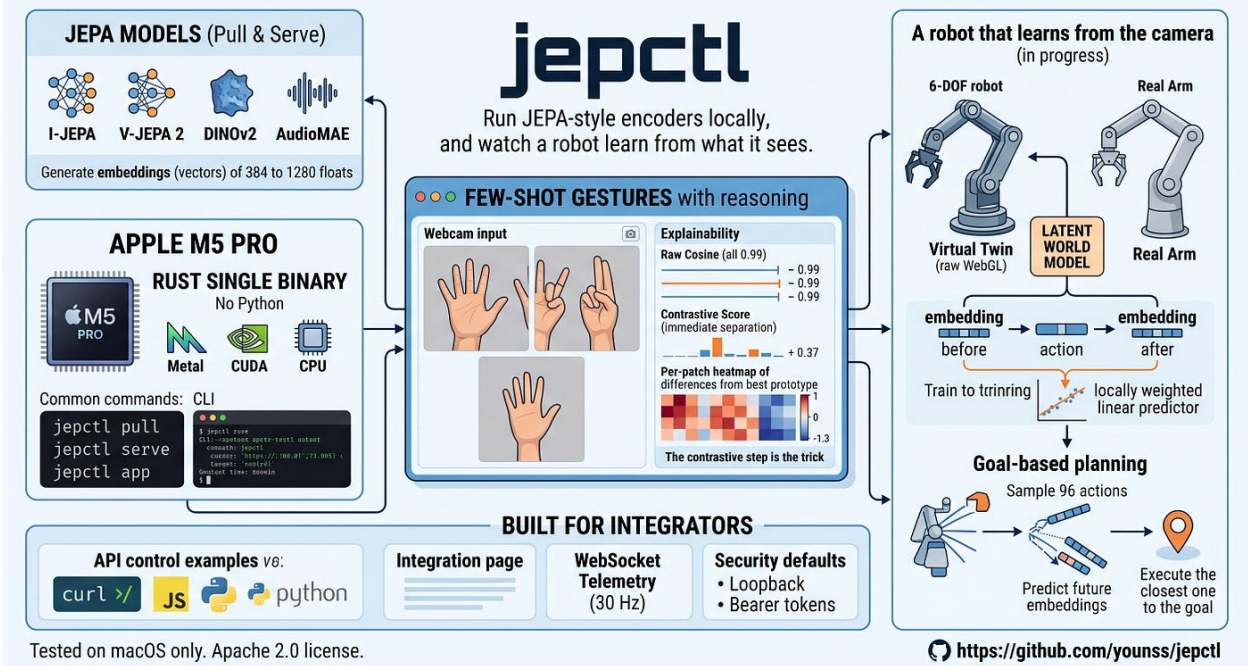


Figure 1: Overview of `jepctl`. Left: catalogued JEPA-style encoders (I-JEPA, V-JEPA 2, DINOv2, AudioMAE) producing embeddings of 384 to 1280 dimensions, served by a single Rust binary with Metal, CUDA, or CPU backends and a small command line. Centre: few-shot recognition with its reasoning exposed, including the raw cosine scores, the contrastive score that separates otherwise near-identical classes, and the per-patch difference map. Right: the camera-driven robot loop, in which embeddings before and after an action train a locally weighted linear predictor that is then used to plan by sampling candidate actions and executing the one predicted to land closest to the goal. Bottom: the integration surface and the security defaults. Figure by the author.

`jepctl` is that missing runtime. It is a single Rust binary with no Python dependency that (a) downloads and loads JEPA-style checkpoints with an explicit, auditable report of how many tensors were covered, (b) serves embeddings over HTTP, a live SSE stream, and a command line, and (c) ships a desktop testbench in which four applications are built entirely out of latent-space operations.

The contribution of this paper is not a new architecture or a new training objective. It is a system, and three claims about it:

1. **Operational honesty is a feature, not an implementation detail.** Loading a checkpoint partially yields a model that still returns vectors of the right shape and is therefore indistinguishable from a working one at the API level. `jepctl` refuses to serve such a model and reports coverage explicitly (Section 3.1).
2. **A frozen encoder plus a small online predictor is enough to demonstrate the world-model loop on a laptop.** We follow the structure of LeWorldModel [10] — predict the next embedding, read the prediction error as surprise — but freeze the encoder and fit the predictor online on the live stream, which makes the loop observable in seconds without a training run (Section 5).
3. **Several economically interesting behaviours need no training at all.** Few-shot in-

dustrial inspection, skeleton-free teleoperation, and reward-free visual goal reaching are all nearest-prototype or energy-minimisation queries in a pretrained latent space (Sections 4–6).

2 Background and related work

Joint-embedding predictive architectures. A JEPA encodes an observation o into a latent $z = f_\theta(o)$ and trains a predictor to map a context latent to the latent of a target that was masked or displaced in time, minimising a distance in embedding space rather than in pixel space [9]. I-JEPA applies this to masked image regions [1]; V-JEPA 2 extends it to video with spatio-temporal tubelets and demonstrates planning [2]. DINOv2, while trained with a different self-distillation objective, produces patch-level features with the same practical property we rely on here: semantic structure that is stable under nuisance variation [12].

LeWorldModel. Maes et al. [10] address the central fragility of JEPA training — collapse to a constant representation — with a two-term objective: a mean-squared next-embedding prediction loss, and SIGReg, a regulariser that projects embeddings onto random directions and applies a normality test to each one-dimensional projection, pushing the latent distribution toward an isotropic Gaussian. The result trains stably end to end from pixels without stop-gradients, exponential moving averages, or a pretrained encoder, with 15M parameters on a single GPU. Two of their findings are directly load-bearing for this work. First, the predictor is autoregressive in latent space: z_{t+1} is predicted from z_t and the action a_t . Second, the resulting prediction error is a usable *surprise* signal: the model reliably flags physically implausible events. Our World module (Section 5) implements that loop with a frozen encoder, and our robot goal-reaching module (Section 6) plans by rolling out a learned latent transition model, in the spirit of their cross-entropy-method planner [13].

JEPA-Anything. Cui et al. [3] argue the complementary point: that the JEPA principle is domain-agnostic. Their orthogonal predictive factorization decomposes latent targets into complementary factors learned through dedicated pathways and recombined in a shared predictive design, validated across vision, biology, clinical trajectories, control, molecular dynamics, physical fields, and weather. Their result motivates a runtime whose abstractions are modality-agnostic: `jepctl` treats an image, a video clip, and an audio spectrogram as the same object — an input that becomes an embedding — and its few-shot matcher, its world model, and its control loops are written against embeddings, not against pixels. We do not implement OPF; we take from that work the design constraint that nothing in the runtime above the encoder should assume a modality.

What this is not. JEPA encoders are not generative. They do not decode a latent back to pixels, and they do not estimate metric depth. Throughout this paper, and in the software’s own interface, we are explicit about that boundary; Section 9 states it as a limitation rather than hiding it behind a visualisation.

3 System design

`jepctl` is written in Rust and builds to one binary. Inference uses `candle` [7] with Metal, CUDA, or multi-threaded CPU backends selected at runtime. The HTTP layer is `axum`; the desktop testbench is a native window hosting a dependency-free web UI. Figure 1 gives the overall picture and Figure 2 the dataflow.

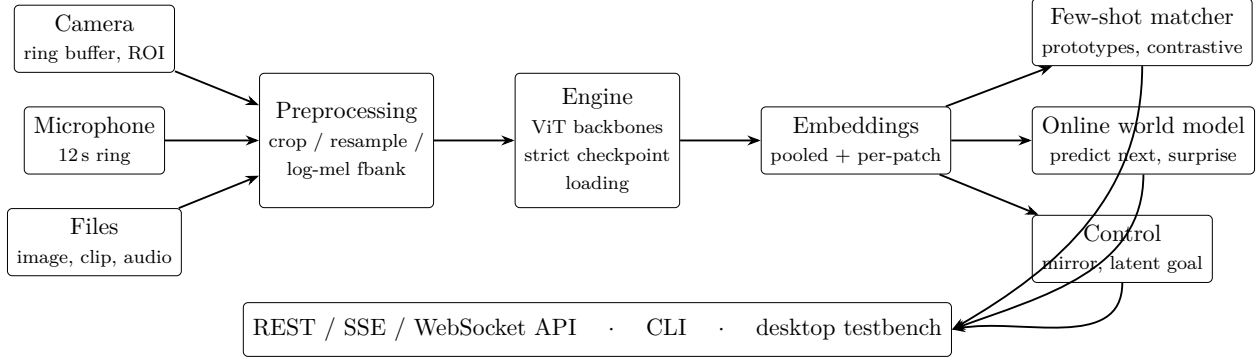


Figure 2: `jepctl` architecture. Every module above the encoder operates on embeddings only; none of them reads pixels. The same path serves the command line, the HTTP API, and the desktop testbench.

Checkpoint	Family	Dim	Coverage
facebook/ijepa_vith14_1k	I-JEPA ViT-H/14	1280	516/516
facebook/dinov2-small	DINOv2 ViT-S/14	384	221/221
facebook/vjepa2-vitl-fpc64-256	V-JEPA 2 ViT-L	1024	388/388
google/vit-base-patch16-224	ViT-B/16	768	197/197
timm ViT-B/16 (fused QKV)	ViT-B/16	768	197/197
gaunernst/vit_base_patch16_1024_128.audiomae_as2m	AudioMAE ViT-B	768	197/197

Table 1: Checkpoint coverage reported by `jepctl` for the catalogued encoders. Coverage is the number of backbone parameters populated from the checkpoint over the number the architecture declares; anything below 100 % is refused at load time.

3.1 Strict checkpoint loading

A partially loaded Vision Transformer [4] is a silent failure: the forward pass succeeds, the output has the advertised dimension, and every downstream metric is meaningless. `jepctl` therefore maps every parameter of the constructed backbone against the checkpoint, trying the naming conventions of Hugging Face ViT/I-JEPA, Hugging Face DINOv2, Hugging Face V-JEPA 2, and timm-style fused QKV, and reshaping tensors whose element count matches (for instance a `Conv3d` patch kernel feeding a linear tubelet embedding). It then returns a coverage report. A model whose coverage is below 100 % is refused with HTTP 422 rather than served. Positional embeddings are adapted separately: the CLS row is sliced or synthesised as required, and the patch grid is resampled bicubically when the checkpoint resolution differs from the configured one.

Weights are read from `safetensors` [6], which is parsed and memory-mapped but never executed; this removes the arbitrary-code-execution surface of pickle-based checkpoints. Table 1 lists the coverage we obtain for the catalogued encoders.

3.2 Modality-agnostic input path

Images are centre-cropped and resized to the model input with the normalisation declared by the manifest (ImageNet or Inception statistics). Video clips are decoded natively for GIF and animated WebP and through an external `ffmpeg` binary for MP4/WebM, then uniformly sub-sampled to the clip length the model expects; image models embed each sampled frame and average. Audio is down-mixed to mono, resampled to 16 kHz, and turned into a Kaldi-style log-mel filterbank of

1024 × 128 for the AudioMAE front end [5]. Because that encoder’s CLS token was never trained as a summary, the manifest selects mean pooling; using CLS yields near-identical vectors for every input, which is exactly the kind of silent failure the coverage report is designed to prevent.

3.3 Deployment posture

The daemon binds 127.0.0.1 by default and can be switched to all interfaces from a settings toggle, in which case authentication is mandatory. Bearer tokens are stored as SHA-256 digests and compared in constant time; no CORS headers are emitted by default, so a web page open in the user’s browser cannot read camera frames or embeddings. Decoders are bounded (8192 px per side, 256 MB of decoder allocation, 512 frames or 256 million pixels per animation) so that a decompression bomb is refused before it is allocated.

4 Few-shot perception in latent space

The first application registers a small number of reference embeddings per class and classifies by similarity. Let $\{z_i^{(c)}\}$ be the ℓ_2 -normalised embeddings captured for class c and p_c their normalised mean. For a query z , a naive matcher scores $\cos(z, p_c)$. That score is dominated by whatever the classes share — the room, the lighting, the operator’s torso. `jepctl` therefore also computes a contrastive score in the centroid-removed subspace: with $\bar{p} = \frac{1}{C} \sum_c p_c$, it scores $\cos(z - \bar{p}, p_c - \bar{p})$, and reports a blend $s_c = 0.35 \cos(z, p_c) + 0.65 \cos(z - \bar{p}, p_c - \bar{p})$, gated by a threshold and a minimum lead over the runner-up. A dedicated *neutral* class absorbs frames in which nothing intentional is present, which is what prevents an empty scene from being forced into the nearest class.

Localising the deviation. Because the encoder emits one vector per patch, the same machinery localises *where* a query departs from its prototype: the per-patch distance to the best-ranked class’s patch prototype, arranged on the 16 × 16 ViT grid, is a divergence map. In the industrial-inspection scenario shipped with the tool — a bottle, capped or not — two short captures are enough to separate the two states, and the divergence map concentrates on the neck. Nothing was ever trained to look for a cap; the localisation is a consequence of patch-level semantics. This is the property that makes the approach economically interesting: the classical alternative requires a labelled dataset and a training run per defect class.

Why latent space rather than pixel differencing. Glass and plastic produce specular highlights that move with the operator and the ambient light. Pixel differencing responds strongly to those high-frequency luminance changes; the patch embeddings we use are comparatively stable under them, because the pretraining objective rewards representations that are predictable across nuisance variation. We report this as an observed qualitative behaviour of the shipped scenario, not as a quantitative robustness claim.

5 An online latent world model over the live camera

The World module implements the LeWorldModel loop [10] with one deliberate substitution: the encoder is frozen (any catalogued JEPA encoder) and only the predictor is fit, online, on the live stream. This trades the representation-learning contribution of [10] — which their SIGReg term exists to protect — for something a user can watch converge in seconds on a laptop, with no training run and no checkpoint to distribute. Figure 3 shows the loop.

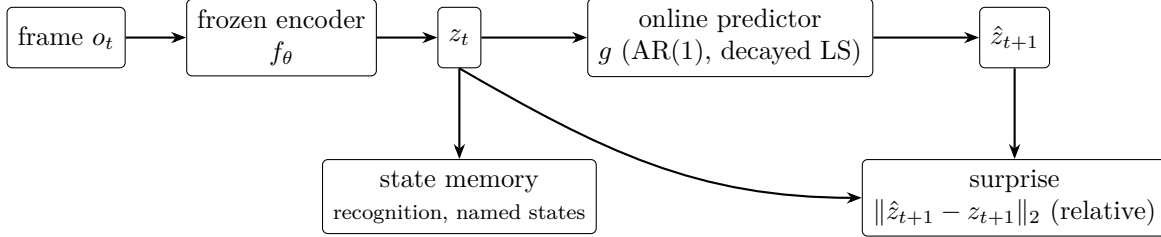


Figure 3: The World loop. The encoder is frozen; the predictor is fit online from the stream. The prediction error of the previous step, normalised by the recent typical error, is the surprise signal used for anomaly detection, following [10].

Predictor. The pooled embedding is projected onto a fixed random basis of dimension $K = 48$; by the Johnson–Lindenstrauss lemma [8] this preserves distances well enough for the error statistic we need while keeping the online fit cheap and stable. In that space a per-dimension first-order autoregression $x_{t+1} = a \odot x_t + c$ is fit by decayed least squares (decay 0.97), falling back to persistence until enough transitions have accumulated.

Surprise. Let $e_t = \|\hat{z}_t - z_t\|_2$ be the error of the prediction made at $t - 1$, and $\bar{e}$ an exponentially weighted estimate of the recent typical error. The reported surprise is $\text{clip}((e_t/\bar{e} - 1)/1.5, 0, 1)$, so that a scene the model tracks well sits near zero and a genuine deviation spikes. This is the operationalisation of the surprise evaluation in [10]: prediction error in latent space, not in pixel space, is what flags the implausible event. Per patch, the same comparison against a constant-velocity prediction $2z_t - z_{t-1}$ yields a spatial surprise map, which the interface renders as a glow on the diverging region and which drives a configurable alarm with a timestamped, spatially localised incident log.

Recognition and named states. A bounded memory of normalised pooled embeddings gives a recognition score (nearest neighbour by cosine), and novel states are appended, so the system reports whether the current view is familiar. Users may additionally save named reference states (“empty desk”, “full shelf”), after which the module reports which one it sees or flags the view as unknown.

Rendering the perception, honestly. The reconstruction shown to the user maps the live frame onto a surface whose relief is derived from the semantic distance of each patch to a background prototype (the mean of the patches whose distance to the frame centroid is below the median). A smoothstep transfer keeps the background flat and extrudes only the foreground, and the patch grid is upsampled bicubically so silhouettes are smooth. We label this in the interface as what it is: the real frame given shape by what the encoder perceives. It is neither a generated image nor a metric depth map.

6 Control without trajectories or rewards

The robot module drives a 6-DoF arm with a gripper through a hardware abstraction trait with two implementations: an in-memory backend feeding a WebGL twin, and a serial backend speaking a Feitech STS3215 register protocol. A safety supervisor enforces per-joint limits, a 1.5 rad/s velocity ramp at 30 Hz, and an emergency stop that only an administrator can reset; with the physical

backend every command is held until explicitly approved. Two control modes are latent-space operations.

Mirror: teleoperation without skeleton tracking. The operator demonstrates a handful of postures, each associated with an arm pose. At run time the arm does not snap to the winning class; it moves to the softmax-weighted blend of the demonstrated poses, with weights $w_c \propto \exp((s_c - \max_j s_j)/\tau)$, $\tau = 0.06$, candidates more than 0.20 below the best receiving zero weight and the blend suppressed entirely below a score of 0.35. Raising an arm slowly therefore sweeps the joint targets continuously through the demonstrated poses. Unlike pose-estimation pipelines, no joint of the human body is ever localised: the comparison is between whole-posture embeddings.

Visual goal reaching: energy minimisation in latent space. Following the planning structure of [10], the arm captures a goal embedding $z_\star$ from the camera and then minimises

$$E(z, z_\star) = \frac{\|z - z_\star\|_2}{\sqrt{d}}, \quad (1)$$

where d is the embedding dimension. A latent transition model is fit online from exploration: triples (z, a, z') collected while the arm babbles are used to fit a locally weighted ridge regression $\Delta z = W[a; 1]$, refit after every observation. At each step the controller samples 96 candidate actions, predicts their outcome *inside the learned model*, executes the one with the lowest predicted energy plus exploration noise, observes the real result, and learns from it. Until enough transitions exist the policy is random, and the telemetry reports which regime is active together with predicted versus observed energy, so the user can judge the model rather than trust it. Convergence is declared against a noise floor measured from two observations of a static scene, and a plateau detector reports when a single viewpoint cannot disambiguate further.

Because the virtual arm is invisible to a real camera, the observation used in simulation is the camera frame with the twin’s kinematics rendered into it over a frozen background, so that the only thing changing between observations is the arm itself. This is a simulation convenience and we flag it as such; with the physical backend the observation is the raw camera frame.

7 Multimodal companion

The companion is a virtual character that fuses two frozen encoders locally. Motion attention is computed from the centroid of the thresholded difference between consecutive 32×24 grayscale thumbnails at 10 Hz and steers the head; visual cues are matched at roughly 3 Hz against the gesture prototypes; audio cues are matched at roughly 1.4 Hz from the last 1.5 s of microphone audio, zero-padded into the AudioMAE window. Recognised cues fire behaviours with a two-second cooldown, and postures mapped to body poses are blended by the same softmax rule as the arm. A 30 Hz control loop renders the result. The point of this module is the absence of the usual machinery: there is no transcription model, no keyword spotter, and no object detector — a clap and a wave are both nearest-prototype lookups over local embeddings.

8 Measurements

The numbers below were obtained on a single machine (Apple M5 Pro, 15 logical cores, 24 GB unified memory, macOS, Metal backend) during interactive use. They characterise this implementation on this hardware; they are not benchmarks and no baseline comparison is claimed.

Quantity	Setting	Observed
World endpoint latency	DINOv2-S, full frame, relief and texture	12–46 ms per call
Gesture stream latency	DINOv2-S, model view	12–15 ms per frame
V-JEPA 2 clip forward	ViT-L, 16 frames at 256 px, peak RSS	1.98 GB
Daemon under load	V-JEPA 2 loaded, SSE stream and 4 concurrent captures	1.5 GB peak
Goal reaching (simulated arm)	after ≈ 45 s of exploration	all 6 joints within ≈ 0.1 rad in 15–30 s
Latent transition model fit	simulated arm, frozen background	fit error ≈ 0.05

Table 2: Single-machine measurements.

A memory bug worth reporting. Every model was originally constructed from a parameter map whose tensors were autograd-tracked variables, because that is the natural way to copy a checkpoint into a pre-built architecture. Inference through such tensors records a backward graph, and that graph keeps every intermediate activation of every layer alive until the output is dropped. Instrumenting the Metal allocator showed GPU allocation climbing linearly by roughly 0.5 GB per transformer layer, reaching 13.5 GB after 24 layers for a single V-JEPA 2 clip; with concurrent requests the process reached 36 GB and was killed. Rebuilding each model on detached tensors after loading — storage is shared, so nothing is copied — reduced the peak resident set for one clip forward to 1.98 GB, of which 1.2 GB is the weights themselves. We report this because the failure mode is invisible in a framework that assumes training: nothing is wrong except that inference is paying for a gradient nobody will ever use.

Correctness practices. The repository contains 76 Rust unit and integration tests, including in-memory HTTP tests of every API surface, and 4 UI tests. What has *not* been done is a numerical comparison against the reference PyTorch implementations, because the project has no Python dependency. Implementations follow the reference code structure, including V-JEPA 2’s tiled sine/cosine rotary tables with interleaved rotation and the Kaldi Povey window for the audio front end, and determinism and discriminativeness are checked, but a bit-level equivalence claim would be unsupported and we do not make one.

9 Limitations

- **Not generative, no metric depth.** JEPA encoders do not decode latents to pixels and do not estimate distance. The 3D reconstruction in the World module is the real camera image given relief by semantic foreground separation; calling it depth would be wrong.
- **The online predictor is weak by construction.** A per-dimension AR(1) fit in a 48-dimensional random projection is adequate to expose the predict-then-measure-surprise loop and to detect deviations, but it is not the learned predictor of [10] and makes no claim to their control performance.
- **Single viewpoint.** Goal reaching can plateau at a pose that is view-equivalent to the goal; the system reports the plateau rather than pretending to converge.
- **Few-shot means few-shot.** Prototypes interpolate between what was demonstrated and do not extrapolate; recognition degrades under large changes of viewpoint, scale, or illumination relative to the captures.

- **No published Audio-JEPA checkpoint.** The audio path is complete, but the verified audio encoder is AudioMAE [5]; a JEPA-trained audio encoder in a loadable format would be a manifest entry away.
- **The serial robot backend is frame-tested only.** The STS3215 protocol is implemented from the register map and unit-tested at the frame level; it has not been run against physical hardware by the authors.

10 Conclusion

The JEPA literature has moved quickly from a training principle [9] to stable end-to-end world models [10] and to a claim of domain generality [3]. `jepctl` is an attempt to close the distance between that literature and a practitioner with a laptop and a camera: pull a checkpoint, be told exactly how much of it loaded, and then reach for recognition, anomaly detection, or control as operations on embeddings. The applications it ships are deliberately ordinary — inspect a bottle, mirror a posture, reach a pictured goal, react to a clap — because the interesting claim is not that any of them is novel, but that each of them costs a handful of captures instead of a dataset, and runs entirely on the machine in front of you.

Availability

Source code, documentation, and the model catalogue are available under the Apache-2.0 licence at <https://github.com/younss/jepctl>. The measurements in Section 8 correspond to version 0.3.0.

Acknowledgements

The author thanks the maintainers of the open-source checkpoints catalogued by this tool.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023.
- [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohov, Sergio Arnaud, Abha Geji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [3] Taoyong Cui, Zhongyao Wang, Xinyue Xu, Weiyang Liu, Zhaochen Yu, Yuying Zhang, Qiang Gao, Mengyue Yang, Wanli Ouyang, Pheng Ann Heng, Yingcheng Wu, Zhenfei Yin, and Ling Yang. JEPA-Anything: Learning predictive models across different worlds. *arXiv preprint arXiv:2609.20800*, 2026.

- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [5] Po-Yao Huang, Hu Xu, Juncheng Li, Alexei Baevski, Michael Auli, Wojciech Galuba, Florian Metze, and Christoph Feichtenhofer. Masked autoencoders that listen. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] Hugging Face. Safetensors: a simple, safe format for storing tensors. <https://github.com/huggingface/safetensors>, 2023.
- [7] Hugging Face. Candle: a minimalist ml framework for rust. <https://github.com/huggingface/candle>, 2024.
- [8] William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.
- [9] Yann LeCun. A path towards autonomous machine intelligence. Technical report, OpenReview, 2022. Version 0.9.2.
- [10] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- [11] Ollama. Ollama: get up and running with large language models locally. <https://github.com/ollama/ollama>, 2024.
- [12] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [13] Reuven Y. Rubinstein and Dirk P. Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.